

# Adapter

## 1. Explicación del patrón

Adapter es un patrón estructural que permite que dos clases con interfaces incompatibles puedan colaborar entre sí.

Su objetivo es transformar una interfaz en otra para que una clase pueda utilizar a otra a través de la interfaz que espera.

La idea principal es crear una clase intermedia, llamada adaptador, que actúa como traductor entre el cliente y la clase que se quiere reutilizar.

El cliente no llama directamente a la clase incompatible. En su lugar, llama al adaptador usando una interfaz conocida. Después, el adaptador traduce esa llamada y se la pasa al objeto adaptado en el formato que este entiende.

De esta forma, se puede reutilizar una clase existente aunque su interfaz no coincida con la que necesita el sistema.

También puede entenderse como un envoltorio: el adaptador envuelve al objeto incompatible y oculta al cliente los detalles de conversión.

## 2. Cuándo se utiliza

Se utiliza el patrón Adapter cuando:

- Se quiere usar una clase existente, pero su interfaz no concuerda con lo que se necesita.
- Se quiere reutilizar una clase cuya interfaz no es compatible con el código actual.
- El cliente espera una interfaz concreta y el objeto disponible ofrece otra distinta.
- No se puede modificar la clase existente o no conviene hacerlo.
- Se trabaja con clases heredadas, externas o de terceros que tienen una interfaz diferente.
- Se quiere separar la lógica de conversión de la lógica principal del programa.
- Se quiere que el cliente siga usando su interfaz habitual aunque internamente se use otro objeto.
- Se necesita adaptar llamadas, parámetros, formatos de datos o nombres de métodos.
- Se quiere introducir una clase intermedia sin cambiar el código cliente existente.

## 3. Cómo aplicarlo

Para aplicar Adapter se suelen definir estos participantes:

### Cliente

Es la clase que necesita usar una funcionalidad.

El cliente trabaja con una interfaz concreta y espera que los objetos cumplan esa interfaz.

El cliente no debe conocer directamente al objeto incompatible.

# Interfaz objetivo

Es la interfaz que el cliente espera utilizar.

Define los métodos que el cliente sabe llamar.

Ejemplo conceptual:

- `solicita()`

En el ejemplo de coches del material:

- `IMandos`
- `acelera()`
- `frena()`

## Adaptado

Es la clase existente que se quiere reutilizar.

Tiene funcionalidad útil, pero su interfaz no coincide con la que espera el cliente.

Ejemplo conceptual:

- `realiza()`

En el ejemplo de coches:

- `CocheManual`

En el ejemplo de barco:

- `Barco`

## Adaptador

Es la clase que conecta la interfaz esperada por el cliente con la interfaz real del objeto adaptado.

Responsabilidades:

- Implementar la interfaz objetivo.
- Mantener una referencia al objeto adaptado.
- Recibir llamadas del cliente.
- Traducir esas llamadas a métodos del objeto adaptado.
- Convertir datos, parámetros o formatos si es necesario.
- Delegar el trabajo real en el objeto adaptado.

Ejemplo conceptual:

El cliente llama a:

```
adaptador.solicita()
```

y el adaptador internamente llama a:

```
adaptado.realiza()
```

## 4. Forma práctica de aplicarlo

Pasos:

1. Identificar una clase que se quiere reutilizar pero que tiene una interfaz incompatible.
2. Identificar qué interfaz espera el cliente.
3. Crear una interfaz objetivo con los métodos que el cliente necesita.
4. Crear una clase adaptadora que implemente esa interfaz objetivo.
5. Añadir al adaptador una referencia al objeto adaptado.
6. Inicializar el adaptado dentro del adaptador o pasarlo por constructor.
7. Implementar los métodos de la interfaz objetivo en el adaptador.
8. Dentro de esos métodos, traducir la llamada del cliente a llamadas compatibles con el adaptado.
9. Hacer que el cliente use el adaptador en lugar de usar directamente la clase incompatible.
10. Si aparece otra clase incompatible, crear otro adaptador sin modificar el código cliente.

## 5. Idea clave

El cliente no debe adaptarse a la clase incompatible.

Se crea un adaptador para que la clase incompatible parezca compatible con la interfaz que el cliente espera.

Así se reutiliza código existente sin cambiar el cliente ni modificar la clase adaptada.

## 6. Ejemplos del material

### Ejemplo general

En el diagrama general del material aparecen estos elementos:

- Cliente
- Interfaz
- Adaptador
- Adaptado

El cliente utiliza la interfaz y llama al método `solicita()` .

El adaptador implementa esa interfaz y traduce la llamada hacia el adaptado, que ejecuta su propio método `realiza()` .

La relación práctica sería:

```
Cliente -> Interfaz -> Adaptador -> Adaptado
```

El cliente cree que está trabajando con la interfaz esperada, pero realmente el adaptador está delegando el trabajo en el objeto adaptado.

## Ejemplo de pago externo

En el material aparece un ejemplo relacionado con un servicio de pago externo.

Elementos principales:

- `SolicitudExterna`
- `RespuestaExterna`
- `ServicioPagoExterno`
- `AdaptadorPagoExterno`
- `IProcesadorPago`
- `Pago`
- `ResultadoPago`

La idea es que el sistema interno trabaja con una interfaz propia, por ejemplo `IProcesadorPago` .

Sin embargo, el servicio externo usa sus propias clases y métodos, como `SolicitudExterna` , `RespuestaExterna` y `hacerPago()` .

El adaptador se encarga de convertir un `Pago` interno en una `SolicitudExterna` , llamar al servicio externo y transformar la `RespuestaExterna` en un `ResultadoPago` .

Así, el sistema puede usar el servicio externo sin acoplarse directamente a su interfaz.

## Ejemplo de coche manual y mandos

En el material aparece un ejemplo con coches.

La interfaz esperada es:

- `IMandos`

Con operaciones:

- `acelera()`
- `frena()`

El coche manual tiene su propia forma de trabajar, con atributos y operaciones como:

- `revoluciones`
- `velocidad`
- `marcha`
- `acelera()`
- `subeMarcha()`
- `bajaMarcha()`
- `frena()`

El adaptador permite controlar el coche manual a través de la interfaz `IMandos` .

El cliente puede pedir acelerar o frenar, y el adaptador se encarga de traducir esas acciones al funcionamiento real del coche manual.

## Ejemplo de coche y barco

En el material también aparece una extensión con un barco.

La interfaz sigue siendo:

- `IMandos`
- `acelera()`
- `frena()`

El barco tiene una interfaz distinta, con operaciones como:

- `arrancar()`
- `activar()`
- `moverMasRapido()`
- `detener()`
- `desconectar()`

Para que el barco pueda usarse mediante la interfaz `IMandos`, se crea un `AdaptadorBarco`.

Así, el cliente puede manejar objetos distintos, como coche o barco, usando los mismos mandos.

## 7. Ejemplo conceptual de uso

Sin Adapter, el cliente tendría que conocer la interfaz concreta del objeto incompatible.

Por ejemplo:

```
barco.moverMasRapido()
```

Pero si el cliente espera trabajar con `IMandos`, puede usar:

```
mando.acelera()
```

El adaptador recibe `acelera()` y lo convierte internamente en la llamada adecuada para el barco.

De esta forma, el cliente no necesita saber si por debajo hay un coche manual, un coche automático o un barco.

## 8. Ventajas prácticas

- Permite reutilizar clases existentes con interfaces incompatibles.
- Evita modificar código heredado o externo.
- Reduce el acoplamiento entre el cliente y la clase adaptada.
- Separa la conversión de interfaces de la lógica principal.
- Permite introducir nuevos adaptadores sin cambiar el cliente.
- Facilita trabajar con librerías externas o clases antiguas.
- Hace que objetos distintos puedan usarse mediante una interfaz común.

## 9. Cuidado

Adapter añade una clase intermedia, por lo que puede aumentar la complejidad del diseño.

Si se puede modificar directamente la clase existente para que implemente la interfaz esperada, quizá no sea necesario crear un adaptador.

También hay que evitar que el adaptador acumule demasiada lógica. Su responsabilidad principal debe ser traducir llamadas y adaptar interfaces, no resolver toda la lógica de negocio.

## 10. Diferencia práctica con otros patrones estructurales

Adapter cambia la interfaz con la que se accede a un objeto existente.

Decorator mantiene la misma interfaz o la amplía para añadir responsabilidades.

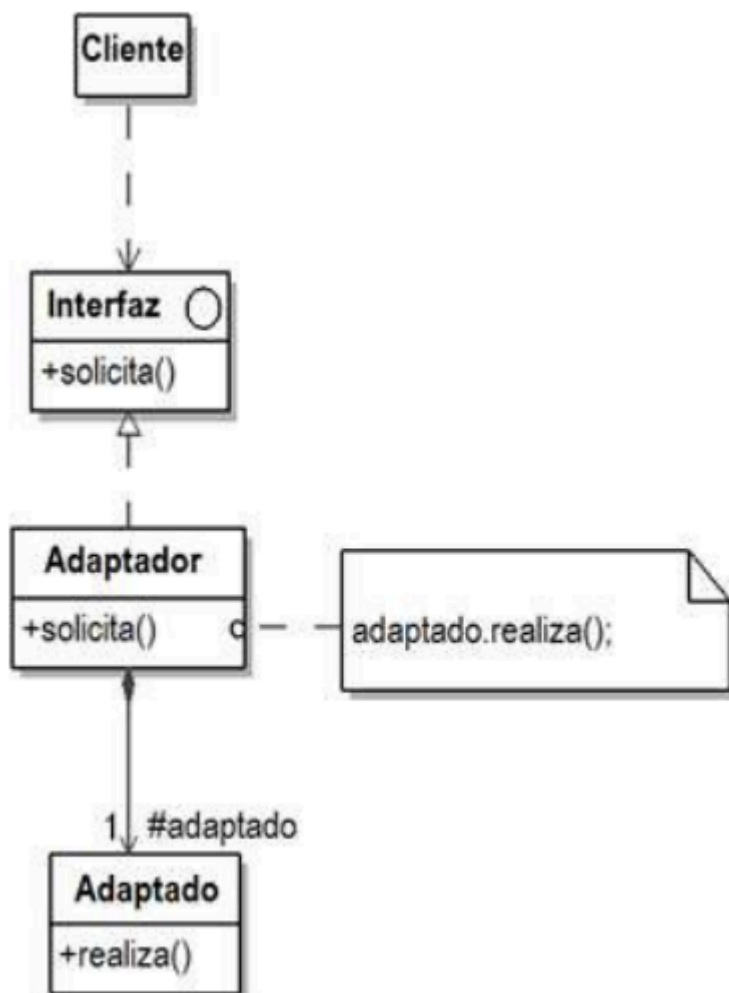
Proxy mantiene la misma interfaz para controlar el acceso a un objeto.

Facade proporciona una interfaz más simple para un subsistema completo.

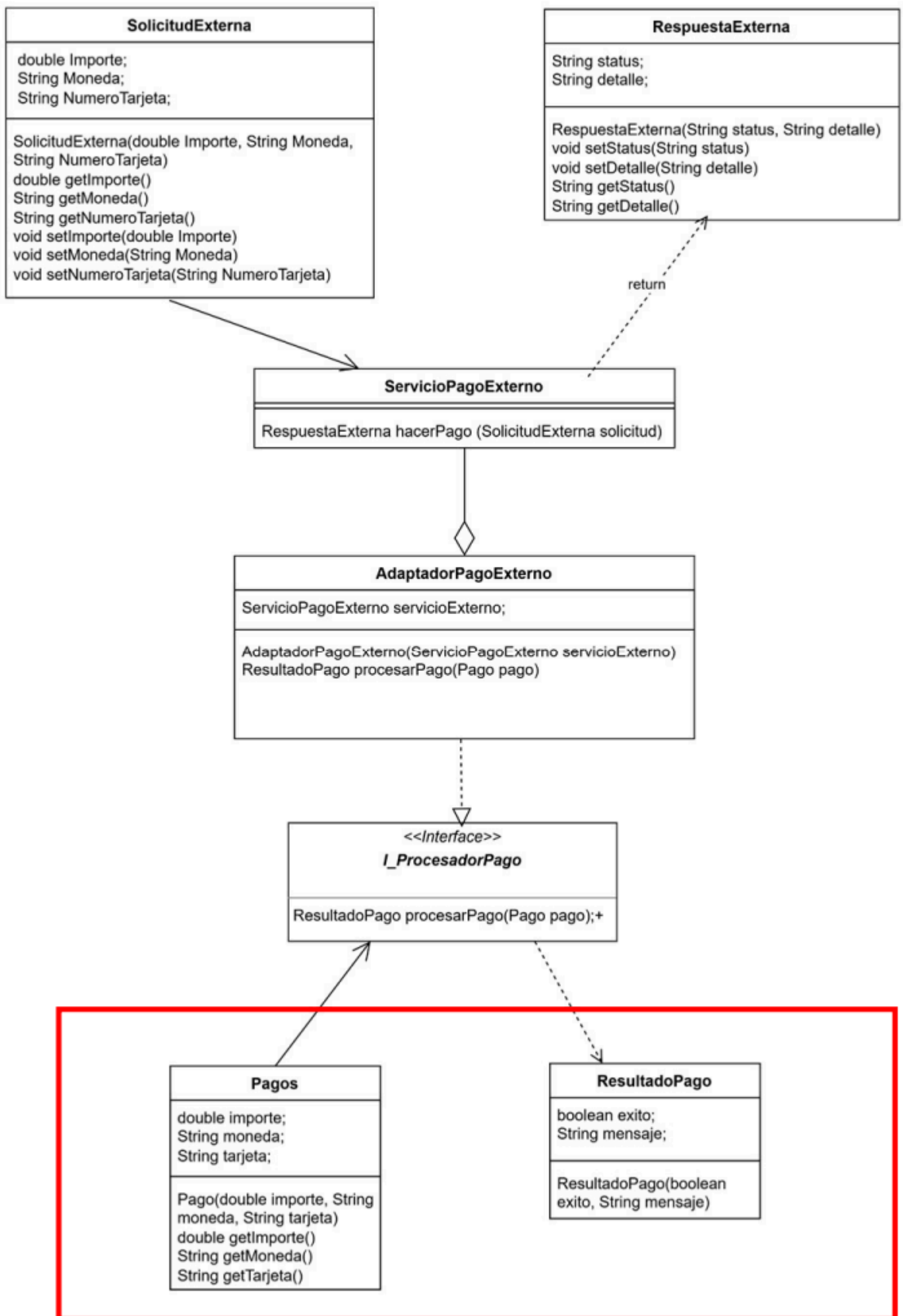
Por tanto:

- Adapter sirve para hacer compatibles interfaces que no encajan.
- Decorator sirve para añadir comportamiento.
- Proxy sirve para controlar el acceso.
- Facade sirve para simplificar el uso de un conjunto de clases.

## 11. Plantilla



## 12. Ejemplo1



## 13. Ejemplo2

